6.1.3 Using can-i-deploy to verify that you can safely deploy to production

As we first mentioned in chapter 4, when you integrate consumer tests into your pipeline, you need to make sure that any code changes from the consumer can be safely deployed to production. The same guideline can also be said for the provider contract tests. If the provider made some code changes and wants to deploy their changes to production, the can-i-deploy tool should also be used on the provider side to deploy these changes confidently and safely.

6.2 Contract testing tooling setup for the provider

As we mentioned already and you first saw in chapter 4, we use the contract testing tool Pact (https://pact.io/) to show how to start with contract testing because this is the current de facto contract testing tool. Because it will be difficult to cover all the programming languages that Pact supports, this chapter focus on how to set up pact-js, which is the JavaScript version.

> **Note** While we've chosen how to write provider contract tests in JavaScript, Pact supports other programming languages as well, such as Java, C#, Ruby, and Python. Specific implementations of Pact with other programming languages are listed at https://docs.pact.io/implementation_guides/.

As the pact-js online documentation is likely to be more up-to-date with API implementation changes, make sure to always check their documentation for more information (https://mng.bz/j0NP). If you've already followed the setup instructions in chapter 4, section 4.2, you don't need to do much setup in this chapter, apart from saving any local changes you have and switching to a new branch called `setup- contract-tests-provider`.

We'll still be using the example GitHub project that we've created, https://github.com/mdcruz/pact-js-example/, that you can download or fork online using Git. If you need additional guidelines on how to fork or